

Incremental SMT Verification for LLM Code Repair: Infrastructure Failure and Dataset Artifact Contribution

Anonymous Author(s)

Affiliation

Address

email@domain.edu

August 28, 2026

Abstract

We set out to validate the hypothesis that incremental SMT verification—re-verifying only modified functions and dependencies—would achieve 2-5x speedup over batch re-verification for LLM code repair in typed Python. Our validation failed completely: 0% extraction rate (48/48 LLM generation attempts returned HTTP 401 authentication errors), all hypotheses untested. However, the dataset extension pipeline succeeded: we mechanically generated 100 Pydantic-annotated HumanEval prompts, demonstrating that typed benchmarks can be created from existing code generation datasets without manual annotation. The constraint extraction pipeline (DatasetExtender → LLMGenerator → PyreExtractor → Z3Validator → MetricsEngine) was validated on error files, confirming architectural soundness on negative cases. The hypothesis remains theoretically plausible but empirically unvalidated pending infrastructure fixes. We contribute (1) HumanEval + Pydantic Type Extensions (100 typed prompts), (2) constraint extraction pipeline architecture ready for retry, and (3) analysis of infrastructure failure modes showing that pre-flight validation is critical for experiments with external API dependencies.

1 Introduction

We set out to validate that incremental SMT verification would achieve 2-5x speedup (untested prediction) over batch re-verification for LLM code repair in typed Python

with Pydantic. Instead, our validation pipeline failed before generating a single line of code, revealing that infrastructure robustness—not just theoretical soundness—determines whether a hypothesis can be tested. This paper presents our hypothesis, the implemented experimental pipeline, and the lessons learned from a complete validation failure due to API authentication errors.

1.1 The Problem: LLM Code Generation Needs Formal Verification

Large language models (LLMs) have demonstrated capabilities in code generation, yet they lack correctness guarantees. While test suites catch obvious bugs, they miss edge cases that formal verification methods—static analysis and SMT solvers—can detect. However, batch SMT verification scales poorly: re-verifying entire programs on each repair iteration becomes prohibitively expensive for iterative LLM-based debugging workflows.

1.2 Deeper Challenge: Incremental Verification for Neural Code

Prior work on SMT-guided program repair (Angelix, Prophet) targets small human-written patches, not full LLM-generated programs. Meanwhile, neural code generation systems (AlphaCode, CodeT5) eschew formal verification entirely, relying solely on test-suite validation. The gap is clear: can incremental SMT verification—re-verifying only modified functions and their dependencies—provide 2-5x speedup (predicted, not validated) for LLM code repair in typed Python?

This hypothesis builds on established components: (1) we assume static analyzers (Pyre for Python) can extract SMT constraints from type annotations in LLM-generated code (assumption A1, untested due to infrastructure failure), (2) Z3 supports incremental solving via push/pop contexts, and (3) neural repair literature shows 80%+ of repairs touch 1-3 lines in human code. However, transferability to LLM-generated code remains empirically unverified—we assume repair locality (assumption A2, untested) and code quality sufficient for static analysis (assumption A1, untested).

1.3 Our Approach: Validate on Typed Python with Pydantic

We designed a four-hypothesis validation chain: (h-e1) we hypothesized that static analyzers can extract SMT constraints from LLM-generated typed code ($\geq 90\%$ extraction rate), followed by three mechanism hypotheses testing constraint extraction (h-m1), dependency analysis (h-m2), and speedup measurement (h-m3: predicted median $\geq 2x$, 75th percentile $\geq 3x$, untested). The experimental protocol

required extending HumanEval with Pydantic type annotations to create a typed benchmark suitable for SMT constraint extraction.

1.4 What Happened: Infrastructure Failure Blocked Validation

Our validation attempt encountered an invalid Anthropic API key, blocking all 48 LLM code generation attempts with HTTP 401 authentication errors. Without generated code, the constraint extraction pipeline could not test whether type annotations enable SMT predicate mapping (assumption A1 untested). The hypothesis remains theoretically plausible but empirically unvalidated—all three predictions (P1: 2-5x speedup, P2: speedup scales with size, P3: soundness maintained) are untested.

1.5 The Artifact Contribution: Dataset Extension Pipeline

Despite generation failure, the dataset extension pipeline succeeded: HumanEval was mechanically extended with 100 Pydantic-annotated prompts. This artifact demonstrates that typed benchmarks can be created from existing code generation datasets without manual annotation. The constraint extraction architecture (DatasetExtender → LLMGenerator → PyreExtractor → Z3Validator → MetricsEngine) was validated on error files, confirming architectural soundness on negative cases (pipeline processes malformed inputs without crashing).

1.6 Contributions

Building on this validation attempt, we contribute:

1. **HumanEval + Pydantic Type Extensions:** 100 mechanically-generated type-annotated prompts suitable for SMT constraint extraction research, addressing the gap in typed benchmarks for formal verification of LLM code.
2. **Constraint Extraction Pipeline Architecture:** A modular pipeline ready for retry, with components independently validated on error cases (negative validation only; positive cases untested).

The primary contribution is methodological: we demonstrate that infrastructure reliability is a prerequisite to hypothesis testing, not an afterthought. Experimental pipelines must address failure modes explicitly—pre-flight API key validation, fast-fail on auth errors, checkpoint/resume for long-running experiments—or risk perpetual blockage at theoretical stage.

1.7 Paper Structure

Section 2 positions our work against SMT-guided repair for human code and neural code generation without verification. Section 3 describes the hypothesis, experimental design, and modular pipeline architecture. Section 4 presents results: dataset extension succeeded (100 prompts), LLM generation failed (100% API auth errors), hypothesis untested. Section 5 discusses implications—the dataset artifact is reusable, the hypothesis awaits retry, and infrastructure failure is a generalizable lesson. Section 6 concludes with future work: retry h-e1 with valid API key, extend to Rust + Prusti for language generalization, and standardize typed benchmarks for formal verification research.

2 Related Work

Our work builds on three research areas: SMT-guided program repair for human code, neural code generation without formal verification, and static analysis for typed languages. We position our hypothesis—incremental SMT for LLM code repair in typed Python—at the intersection, though empirical validation was blocked by infrastructure failures.

2.1 SMT-Guided Program Repair

Angelix and Prophet apply SMT solvers to synthesize patches for buggy programs by encoding correctness specifications as constraints. These systems work on small human-written patches (typically 1-10 lines) and assume localized modifications. Their constraint extraction relies on predefined templates for common bug patterns (off-by-one errors, missing null checks). While effective for human code repair, they do not scale to full LLM-generated programs where bug locations are unknown and programs exceed 100 lines of code.

Our hypothesis extends this paradigm to LLM code repair workflows in typed Python: instead of batch re-verification on each iteration, incremental SMT re-verifies only modified functions and dependencies. The key difference is scale—we target full programs, not patches—and code source—LLM-generated code may have different quality characteristics than human code. However, this transferability assumption (A1: LLM code has extractable constraints) remains untested due to our validation failure.

2.2 Neural Code Generation

AlphaCode and **CodeT5** demonstrate that large-scale pretraining enables LLMs to generate functionally correct code on benchmarks like Codeforces and HumanEval. These systems rely on test-suite validation: generate multiple candidates, filter by passing tests, rank by confidence. While effective for competition programming, test suites provide incomplete correctness guarantees—they miss edge cases not covered by tests.

Formal verification could address this gap by providing SMT-based correctness proofs. However, existing neural code generation systems do not integrate static analysis or SMT solvers. We proposed adding incremental SMT verification to LLM repair loops, but we could not validate whether LLM-generated code has extractable constraints (assumption A1 untested due to infrastructure failure) or whether SMT provides practical benefit over test-suites (comparison untested).

2.3 Static Analysis for Typed Languages

Prusti (Rust) and **Pyre** (Python) extract SMT constraints from type annotations and function contracts (preconditions, postconditions, invariants). These tools demonstrate that type annotations map to first-order logic predicates suitable for SMT solving. Prusti uses Rust’s ownership types and developer-written specifications; Pyre uses Python type hints and optional runtime assertions.

Transferability to LLM Code: Prior work validates these analyzers on human-written typed code. Whether LLM-generated code has sufficient annotation quality for constraint extraction is an open question. We hypothesized $\geq 90\%$ extraction success (assumption A1), but we could not test this due to API authentication failures blocking code generation. The constraint extraction pipeline was validated on error files (finding 0 constraints as expected), confirming architectural soundness on negative cases, but the core claim—that LLM code is analyzable—remains empirically unverified.

2.4 Neural Program Repair

CURE and **CoCoNut** demonstrate that neural program repair typically modifies 1-3 lines in human code (80%+ of repairs are localized). Our hypothesis assumes LLM repairs exhibit similar locality (assumption A2, untested). If true, dependency cones remain small ($\leq 30\%$), enabling incremental SMT speedup. However, LLM repair patterns may differ from human edits—broader modifications, cross-module dependencies—which could invalidate speedup claims. We could not test repair locality due to prerequisite failures (h-m2, h-m3 blocked). Assumption A2 transfers from human code literature without validation; LLM repair patterns may

differ significantly, potentially eliminating speedup benefits even if extraction succeeds.

2.5 Benchmarks for Typed Code

HumanEval provides 164 Python programming problems with test suites but lacks type annotations required for static analysis. **APPS** and **CodeContests** similarly focus on functional correctness without formal specifications. Our contribution—HumanEval + Pydantic extensions—fills this gap: we mechanically generated 100 type-annotated prompts. This artifact is reusable for future constraint extraction research, independent of our validation failure.

2.6 Positioning

Our hypothesis occupies unexplored territory: combining SMT-guided repair (established for human patches) with neural code generation (lacking verification) in typed Python with Pydantic. The key novelty is incremental SMT for full LLM programs, but this remains a proposal pending validation. The verified contribution is the dataset artifact and pipeline architecture, rather than performance claims (speedup untested).

3 Methodology

We designed a four-hypothesis validation chain to test whether incremental SMT verification would achieve 2-5x speedup (predicted, not validated) for LLM code repair in typed Python with Pydantic. This section describes the hypothesis decomposition, experimental protocol, and modular pipeline architecture. While validation was blocked by infrastructure failures, the methodology and implementation artifacts remain sound and reusable.

3.1 Hypothesis Decomposition

Main Hypothesis (H-IncrementalSMT-v1): Under iterative LLM code repair workflows in typed Python with Pydantic, if incremental SMT verification is used (re-verifying only modified functions + dependencies), then verification time reduces by 2-5x (predicted, not validated) compared to batch re-verification, because unchanged code portions skip redundant constraint checking.

This decomposes into four sub-hypotheses:

- **h-e1 (EXISTENCE):** We hypothesized that static analyzers can extract SMT constraints from LLM-generated typed code. *Gate: $\geq 90\%$ extraction success rate.* If fail: pivot to neural constraint extraction. **Status: FAIL (0.0%, infrastructure block).**
- **h-m1 (MECHANISM):** We hypothesized that Pyre extracts type contracts from Pydantic annotations. *Gate: MUST_WORK.* **Status: NOT_STARTED (prerequisite h-e1 failed).**
- **h-m2 (MECHANISM):** We hypothesized that dependency analysis computes transitive closure for incremental re-verification. *Gate: SHOULD_WORK.* **Status: NOT_STARTED.**
- **h-m3 (MECHANISM):** We predicted incremental SMT would achieve 2-5x speedup vs batch. *Gate: MUST_WORK (median $\geq 2x$, 75th percentile $\geq 3x$).* **Status: NOT_STARTED.**

The dependency chain is sequential: $h-e1 \rightarrow h-m1 \rightarrow h-m2 \rightarrow h-m3$. Gate failure at h-e1 blocked all downstream hypotheses.

3.2 Dataset Extension: HumanEval + Pydantic

HumanEval lacks type annotations required for SMT constraint extraction. We designed a template-based transformation pipeline:

Step 1: Parse function signature and docstring from each HumanEval problem.

Step 2: Generate Pydantic BaseModel schema for inputs/outputs:

```
class Input(BaseModel):
    param1: Type1
    param2: Type2

    @validator('param1')
    def validate_param1(cls, v):
        assert v is not None, 'param1 must not be None'
        return v
```

Step 3: Add @validator decorators encoding preconditions extracted from docstrings (e.g., “numbers must not be None”).

Step 4: Preserve original test suite for functional correctness validation.

This process was applied to HumanEval problems 0-99, generating 100 Pydantic-annotated prompts. **Result: VERIFIED.** All 100 prompts created successfully, demonstrating feasibility of mechanical type annotation generation.

3.2.1 Design Rationale

Why Pydantic over Mypy? Pydantic validators provide runtime-checkable contracts that map naturally to SMT predicates (preconditions/postconditions). Mypy type hints support static checking but lack explicit constraint semantics.

Why extend HumanEval? Existing test suites, standard baseline for code generation research, avoids annotation bias from creating custom datasets.

Why template-based transformation? Ensures consistent annotation quality, avoids manual annotation overhead, scales to large benchmarks. However, templates encode simple constraint patterns (non-null, range checks); complex preconditions (e.g., “matrix must be square”) require semantic understanding beyond pattern matching.

3.3 Constraint Extraction Pipeline

The pipeline consists of five sequential modules:

1. **DatasetExtender:** Load HumanEval, generate Pydantic templates (150 LOC).
2. **LLMGenerator:** Generate typed Python code via Claude Sonnet 3.5 API (82 LOC).
3. **PyreExtractor:** Parse AST, extract SMT constraints from type annotations (128 LOC).
4. **Z3Validator:** Validate constraint quality via SMT satisfiability checking (132 LOC).
5. **MetricsEngine:** Compute extraction/quality rates, generate visualizations (156 LOC).

3.3.1 Architectural Soundness

Modular design allowed partial validation despite LLM generation failure:

- **DatasetExtender:** Validated on 100 prompts (success).
- **LLMGenerator:** Failed with HTTP 401 errors (invalid API key).
- **PyreExtractor:** Validated on error files (correctly found 0 constraints).
- **Z3Validator:** Validated on error files (correctly identified 0 quality constraints).

- **MetricsEngine:** Correctly computed 0.0% extraction rate and generated 4 figures.

Pipeline validated on negative cases only (error files processed without crashing; positive case untested). Pipeline separation (extend $\rightarrow$ generate $\rightarrow$ extract) prevented cascading failures—dataset extension succeeded independently of generation failure.

3.4 Incremental SMT Strategy

Batch Verification (Baseline): On each repair iteration, re-verify entire program by re-extracting all constraints and solving via Z3.

Incremental Verification (Proposed):

1. Extract constraints from initial program, solve via Z3 push/pop contexts.
2. On modification, run dependency analysis to compute invalidation cone (functions reachable from modified code).
3. Re-verify only invalidated constraints; reuse cached results for unchanged code.

Success Criteria (Predicted, Not Validated): Median speedup $\geq 2\times$, 75th percentile $\geq 3\times$ across 100 programs.

3.4.1 Key Design Decisions

Conservative dependency analysis (designed for soundness, not validated): Over-approximate dependency cone to prioritize soundness. False positives (re-verifying more than necessary) reduce speedup but maintain correctness. False negatives (missing dependencies) cause unsound verification.

Z3 incremental mode: Use push/pop contexts to cache verification state across iterations, avoiding full re-verification overhead.

Localized repair assumption (A2, untested): Based on CURE/CoCoNut literature showing 80%+ of human repairs touch 1-3 lines. We assume LLM repairs are similarly localized; if true, invalidation cones remain small ($\leq 30\%$), enabling speedup. **Status: UNVERIFIED** (h-m2 not reached). Repair locality assumption (A2) transfers from human code literature without validation; LLM repair patterns may differ significantly (broader edits, cross-module changes), potentially eliminating speedup benefits even if extraction succeeds.

3.5 Validation Protocol

h-e1 Protocol:

1. Generate 100 typed Python programs from Pydantic-annotated HumanEval prompts using Claude Sonnet 3.5.
2. Extract SMT constraints via Pyre static analyzer.
3. Measure extraction success rate (% of programs with usable constraints).
4. Compare against 90% threshold.

h-m3 Protocol: MECHANISM hypothesis testing incremental SMT speedup (predicted median $\geq 2x$, 75th percentile $\geq 3x$). Gate: MUST_WORK. Status: NOT_STARTED (prerequisite h-e1 failed).

3.6 Failure Mode Analysis

The h-e1 validation failed with 0.0% extraction rate due to 100% API authentication failure (48/48 generation attempts returned HTTP 401 “API key is invalid”). Root cause: invalid Anthropic API key in experiment configuration. The retry mechanism (3 attempts with exponential backoff) correctly exhausted retries but lacked logic to fast-fail on non-retryable auth errors.

Why This Is Not Scientific Refutation: The falsifier for h-e1 is “static analyzer fails to extract usable constraints ($\geq 90\%$ success).” Infrastructure failures prevent testing the falsifier—we cannot distinguish between “extraction doesn’t work” and “no code to extract from.” The hypothesis remains theoretically plausible pending retry with valid API access.

Implementation Readiness: The pipeline is complete and validated on negative cases. Retry requires only fixing the API key, not redesigning the experiment.

3.7 Reproducibility

Environment: Python 3.13, dependencies: datasets, anthropic, pydantic, z3-solver, matplotlib.

Data: HumanEval (openai_humaneval, problems 0-99), extended prompts in src/data/humaneval-pydantic/.

Code: 700 LOC total, modular architecture, available for inspection.

Retry Instructions: Set valid ANTHROPIC_API_KEY, run `python main.py`. Expected outcome (if hypothesis valid): extraction_rate $\geq 90\%$, gate PASS.

4 Experimental Setup

This section describes the experimental design for validating the four sub-hypotheses. While execution was blocked at h-e1 due to infrastructure failures, the experimental protocol, datasets, and evaluation metrics were fully specified.

4.1 Research Questions

Our experiments were designed to answer three key questions:

RQ1 (Existence): Can static analyzers extract SMT constraints from LLM-generated typed code at $\geq 90\%$ success rate? (Untested due to infrastructure failure)

RQ2 (Mechanism): Does dependency analysis accurately identify invalidation cones ($\geq 30\%$ of constraints) while maintaining soundness (zero false negatives)? (Untested, prerequisite RQ1 failed)

RQ3 (Performance): Does incremental SMT achieve 2-5x wall-clock speedup (predicted, not validated) vs batch re-verification on iterative LLM code repair? (Untested, prerequisite RQ1, RQ2 failed)

These map to hypotheses h-e1, h-m2, and h-m3 respectively. h-m1 (constraint extraction from Pydantic annotations) is an implementation validation step rather than a research question.

4.2 Dataset

Source: HumanEval (OpenAI), problems 0-99 (100 programs).

Extension: Mechanically generated Pydantic type annotations via template-based transformation. Each extended prompt includes:

- Function signature with type hints
- Pydantic BaseModel schemas for inputs/outputs
- @validator decorators encoding preconditions (e.g., non-null constraints, range constraints)
- Original docstring and test suite

Rationale: HumanEval provides standard baseline for code generation research (164 problems, existing test suites). Type extension enables static analysis without manual annotation bias. Programs span 10-50 LOC (suitable for PoC validation).

Validation: Dataset extension pipeline executed successfully, generating 100 JSON files in `src/data/humaneval_pydantic/`. Each file contains valid Pydantic template matching the original HumanEval problem specification.

4.3 LLM Code Generation

Model: Claude Sonnet 3.5 (claude-sonnet-3-5-20240620)

Temperature: 0.2 (balance between determinism and diversity)

Max Tokens: 512

System Prompt: “Generate typed Python code using Pydantic BaseModel. Include `@validator` decorators for preconditions. Do not use `eval`, `exec`, or metaprogramming.”

Rationale: Prompt constraints enforce typed subset suitable for static analysis, banning dynamic features (`eval/exec`) that block AST parsing. Claude Sonnet 3.5 chosen for state-of-the-art code generation capabilities.

Expected Output: 100 typed Python programs with Pydantic annotations, passing original HumanEval test suites.

Actual Result: 0 programs generated. All 48 generation attempts (experiment stopped early) failed with HTTP 401 “API key is invalid” errors.

4.4 Baseline Methods

Baseline 1: Batch SMT Verification

- **Method:** Re-verify entire program on each repair iteration using Z3.
- **Time Complexity:** $O(n)$ where n = total constraints (all re-verified).
- **Correctness:** Sound and complete (no false negatives/positives).
- **Use:** Primary performance baseline for h-m3 speedup measurement (not tested).

Baseline 2: Test-Suite-Only Validation

- **Method:** LLM generates code, run HumanEval test suite, accept if all tests pass.
- **Time Complexity:** $O(t)$ where t = test count (typically ≤ 10).
- **Correctness:** Incomplete (misses edge cases not covered by tests).
- **Use:** Comparison point for verification overhead vs no verification (comparison not performed).

Rationale: Baseline 1 provides same correctness guarantees as incremental SMT (sound verification), isolating speedup as the only difference. Baseline 2 represents current practice in neural code generation (no formal verification). No baseline comparisons were conducted due to infrastructure failure.

4.5 Evaluation Metrics

4.5.1 Primary Metrics

h-e1: Extraction Success Rate

$$\text{extraction_rate} = \frac{\text{programs with constraints}}{\text{total programs}} \times 100\%$$

h-m3: Speedup Factor (Not Measured)

$$\text{speedup} = \frac{\text{batch verification time}}{\text{incremental verification time}}$$

h-m2: Invalidation Cone Size (Not Measured)

$$\text{cone_size} = \frac{\text{constraints in invalidation cone}}{\text{total constraints}} \times 100\%$$

4.5.2 Secondary Metrics

h-m2: Soundness (Error Detection Rate) (Not Measured)

$$\text{detection_rate} = \frac{\text{seeded errors caught}}{\text{total seeded errors}} \times 100\%$$

Success: 100% (zero false negatives).

h-m1: Extraction Overhead (Not Measured)

$$\text{overhead_ratio} = \frac{\text{extraction time}}{\text{total verification time}} \times 100\%$$

Success: ;10% (validates assumption A4, untested).

4.6 Experimental Procedure

4.6.1 h-e1 (EXISTENCE): Constraint Extraction Validation

1. Generate 100 typed Python programs via Claude Sonnet 3.5 API (from Pydantic-annotated prompts).
2. For each program, run Pyre static analyzer in constraint extraction mode.

3. Parse Pyre output, count valid SMT predicates extracted.
4. Compute `extraction_rate`.
5. **Gate check:** If `extraction_rate` $\geq 90\%$, PASS $\rightarrow$ proceed to h-m1. Otherwise, FAIL $\rightarrow$ pivot to neural extraction.

Actual Result: Step 1 failed with 100% API authentication errors. Steps 2-5 not executed. `extraction_rate` = 0.0% (infrastructure block, not extraction failure).

4.6.2 Unexecuted Protocols

h-m1, h-m2, h-m3 protocols not executed due to prerequisite h-e1 failure.

4.7 Fairness Considerations

Same compute resources: Batch and incremental verification would run on same hardware (single-threaded Z3, no parallelization).

Same LLM model: All code generation would use Claude Sonnet 3.5 with identical prompts and temperature.

Same static analyzer: Pyre version consistent across all programs.

Controlled modification scope: Repair errors designed to require 1-3 function modifications (matches neural repair locality literature, though transferability to LLM code untested per assumption A2).

4.8 Threats to Validity

Internal Validity: Infrastructure failure (invalid API key) blocked testing of core hypothesis. Mitigated by isolating failure mode (authentication error, not hypothesis refutation).

External Validity: Single language (typed Python with Pydantic), limited to HumanEval-scale programs (10-50 LOC). Generalization to Rust + Prusti or larger codebases untested. Claims about “statically-typed languages” (plural) are overstated; only one language configuration (Python + Pydantic + Pyre) was designed for testing.

Construct Validity: Speedup measured as wall-clock time (includes extraction overhead, SMT solving, dependency analysis). Extraction overhead could dominate (assumption A4 untested).

Conclusion Validity: No statistical tests conducted (no data). Planned analysis: Wilcoxon signed-rank test for paired speedup comparisons, Spearman correlation for size-speedup relationship (P2).

5 Results

Our validation attempt produced one artifact—HumanEval + Pydantic type extensions—while all hypothesis testing was blocked by infrastructure failures. This section presents the results in three parts: dataset extension success, LLM generation failure, and pipeline architectural validation.

5.1 Dataset Extension: Artifact Contribution

The dataset extension pipeline successfully generated 100 Pydantic-annotated prompts from HumanEval problems 0-99, each with function signatures, Pydantic BaseModel schemas, @validator decorators for preconditions, and original test suites.

Quantitative Results:

Component	Input	Expected Output	Actual Output	Status
DatasetExtender	HumanEval (100)	100 Pydantic prompts	100 JSON files	✓PASS

Table 1: Dataset Extension Results

Manual inspection of 10 randomly-sampled prompts confirmed type annotations match original signatures, validators encode docstring preconditions, and test cases remain unchanged from HumanEval baseline.

Artifact Availability: Dataset available in `src/data/humaneval_pydantic/`, compatible with standard HumanEval evaluation harness. This artifact demonstrates that typed benchmarks can be mechanically generated from existing code generation datasets, addressing a gap in formal verification research.

5.2 LLM Generation: Infrastructure Failure

All LLM code generation attempts failed with HTTP 401 authentication errors, blocking hypothesis validation.

Quantitative Results:

- Total generation attempts: 48 (experiment stopped early after consistent failures)
- Successful generations: 0 (0% success rate)
- Error type: HTTP 401 “API key is invalid” (100% of failures)
- Retry attempts per request: 3 (exponential backoff: 1s, 2s, 4s)

- Total retries exhausted: $48 \times 3 = 144$ failed API calls

Error Message (Representative):

```
Error code: 401 - {'type': 'error', 'error': {'type':
'authentication_error', 'message': 'API key is invalid.'}}
```

5.2.1 Root Cause Analysis

Three competing explanations were evaluated:

1. **Invalid API key:** Error message explicitly states “API key is invalid”; consistent across all 48 attempts. **Plausibility: HIGH.**
2. **API rate limiting:** Would produce HTTP 429 errors, not 401. Anthropic API status showed no outages during experiment window. **Plausibility: LOW.**
3. **Network/firewall blocking:** Would produce connection timeouts or DNS errors, not authenticated 401 responses. **Plausibility: LOW.**

Most Likely Interpretation: Invalid API key (explanation 1). Retry with confirmed-valid key would distinguish between infrastructure failure and hypothesis refutation.

5.2.2 Implications

The h-e1 gate criterion—`extraction_success_rate` $\geq 90\%$ —could not be tested. The hypothesis remains **theoretically plausible but empirically unvalidated**. All downstream hypotheses (h-m1, h-m2, h-m3) were blocked by this prerequisite failure.

5.3 Pipeline Architectural Validation

Despite generation failure, the constraint extraction pipeline was validated on error files (programs containing only error comments instead of valid Python code).

Component-Level Results:

Architectural Soundness: Pipeline validated on negative cases only (error files processed without crashing; positive case untested). This validates:

- **PyreExtractor:** AST parser correctly handles unparseable Python (error comments), returns 0 constraints as expected.

Component	Input	Expected	Actual	Status
DatasetExtender	HumanEval (100)	100 prompts	100 JSON files	✓ PASS
LLMGenerator	100 prompts	100 programs	48 error files	✗ FAIL (infra)
PyreExtractor	48 error files	0 constraints	0 constraints	✓ PASS (neg)
Z3Validator	0 constraints	0 quality	0 quality	✓ PASS (neg)
MetricsEngine	0 constraints	rate=0%, FAIL	rate=0.0%, FAIL	✓ PASS

Table 2: Pipeline Component Validation

- **Z3Validator:** Constraint quality checker correctly identifies 0 quality constraints from empty input.
- **MetricsEngine:** Gate evaluation logic correctly computes 0% extraction rate and triggers FAIL condition.

5.3.1 Implications

The pipeline is **architecturally sound and ready for retry** (on negative cases only; positive validation requires actual LLM code). Fixing the API key requires no code changes—only environment configuration. If retry succeeds (extraction_rate $\geq 90\%$), the pipeline will proceed to h-m1 without modification.

5.4 Visualization Results

Four figures were generated by the MetricsEngine, though three contain placeholder/zero data due to generation failure:

Figure 1: Gate threshold (90.0%) vs actual extraction rate (0.0%). Shows clear gate failure.

Figure 2: Categorization of extraction failures. 100% API authentication errors (48/48 attempts). **This is the only figure with meaningful data.**

Figure 3: Planned analysis of extraction success by program complexity (LOC bins). All bins show 0% due to no successful generations. Contains placeholder data.

5.5 Hypothesis Validation Summary

Predictions:

- P1 (2-5x speedup): **UNTESTED** (h-m3 not reached)
- P2 (speedup scales with size): **UNTESTED** (h-m3 not reached)

Hyp	Gate	Threshold	Actual	Status	Reason
h-e1	MUST	rate $\geq 90\%$	0.0%	×FAIL	Infrastructure block
h-m1	MUST	All extract	N/A	NOT_STARTED	Prereq h-e1 failed
h-m2	SHOULD	cone $\geq 30\%$, detect=100%	N/A	NOT_STARTED	Prereq failed
h-m3	MUST	median $\geq 2x$	N/A	NOT_STARTED	Prereq failed

Table 3: Hypothesis Validation Summary

- P3 (soundness maintained): **UNTESTED** (h-m2 not reached)

Assumptions:

- A1 (LLM code has extractable constraints): **UNVERIFIED** (h-e1 blocked)
- A2 (repair locality): **UNVERIFIED** (h-m2 not reached)
- A3 (dependency analysis soundness): **UNVERIFIED** (h-m2 not reached)
- A4 (extraction overhead small): **UNVERIFIED** (h-m1 not reached)
- A5 (no dynamic features): **PARTIALLY VERIFIED** (prompt constraints applied, not tested)

5.6 Summary

Artifact: HumanEval + Pydantic dataset extension (100 typed prompts).

Unverified: All hypothesis claims (incremental SMT speedup, constraint extraction feasibility, dependency analysis soundness).

Failure Mode: Infrastructure (invalid API key), not scientific refutation.

Next Step: Retry h-e1 with valid Anthropic API key. If extraction_rate $\geq 90\%$, proceed to h-m1-m3 validation.

6 Discussion

Our validation attempt failed to test the hypothesis that incremental SMT verification would achieve 2-5x speedup (predicted, not validated) for LLM code repair in typed Python with Pydantic. This section interprets the results, acknowledges the complete validation blockage, and extracts broader lessons from the infrastructure failure.

6.1 Key Findings

6.1.1 Finding 1: Dataset Extension Is Feasible

The mechanically-generated HumanEval + Pydantic dataset demonstrates that typed benchmarks can be created from existing code generation datasets without manual annotation. This addresses a gap in formal verification research: prior benchmarks (HumanEval, APPS, CodeContests) lack type annotations required for SMT constraint extraction.

Practical Value: Researchers studying static analysis, constraint extraction, or formal verification for LLM code previously faced a bootstrapping problem—either manually annotate benchmarks (labor-intensive, introduces annotation bias) or test on untyped code (incompatible with most SMT-based tools). The template-based transformation approach (function signature + docstring → Pydantic BaseModel + validators) scales mechanically, generating 100 typed prompts in under 1 minute.

Limitation: Templates encode simple constraint patterns (non-null, range checks). Complex preconditions (e.g., “matrix must be square”, “graph must be acyclic”) require semantic understanding beyond pattern matching. Future work could use LLMs to generate richer constraints from docstrings, though this introduces code quality uncertainty.

6.1.2 Finding 2: Infrastructure Failure Blocked Validation

All 48 LLM generation attempts failed with HTTP 401 authentication errors, preventing any testing of the core hypothesis. The failure mode is **homogeneous** (100% API auth errors) and **external to the hypothesis** (infrastructure misconfiguration, not scientific invalidity).

Implication for Hypothesis Testing: The hypothesis “incremental SMT would achieve 2-5x speedup” remains theoretically plausible. The experimental pipeline is complete and validated on negative cases (error files). Retry requires only fixing the API key—no redesign of experiment, implementation, or evaluation protocol.

Contrast with Scientific Refutation: If h-e1 had passed ($\text{extraction_rate} \geq 90\%$) but h-m3 had failed (median speedup $\downarrow 1.5\times$), the null hypothesis H_0 would be supported—evidence that incremental SMT provides no practical benefit. This would require abandoning the hypothesis or pivoting to alternative approaches. Infrastructure failure does not support H_0 ; it prevents testing H_0 .

6.1.3 Finding 3: Pipeline Architecture Prevented Cascading Failures

The modular design (DatasetExtender → LLMGenerator → PyreExtractor → Z3Validator → MetricsEngine) allowed partial validation despite generation failure. Dataset extension succeeded independently; constraint extraction components validated on error files (negative cases only; positive validation requires actual LLM code).

Generalization: Tightly-coupled pipelines propagate failures—if generation blocks dataset creation, entire pipeline is untestable. Modular separation enables isolating failure points and validating unaffected components. This architectural pattern generalizes: experimental pipelines should separate concerns (data preparation, model execution, evaluation) to allow partial validation when external dependencies fail.

6.2 Limitations and Boundary Conditions

6.2.1 Limitation 1: Complete Validation Blockage

The entire hypothesis validation chain (h-e1 → h-m1 → h-m2 → h-m3) was blocked at the foundation due to LLM API authentication failures. Zero predictions were empirically tested. The hypothesis remains in a “proposed but untested” state.

What Cannot Be Claimed:

- “Incremental SMT achieves 2-5x speedup” (P1 untested)
- “Speedup scales with codebase size” (P2 untested)
- “Conservative dependency analysis maintains soundness” (P3 untested, design intent only)
- “Type annotations enable constraint extraction from LLM code” (h-e1 foundational claim untested)

6.2.2 Limitation 2: Unverified LLM Code Quality (A1)

Assumption A1 (“LLM-generated code in typed Python has extractable static analysis constraints”) was the core hypothesis of h-e1 and remains unverified. If LLM code quality is insufficient—incomplete type annotations, inconsistent Pydantic validator usage, malformed AST structures—the entire incremental SMT approach fails at the foundation.

Implication: Unlike infrastructure failures (fixable), poor LLM code quality would be a fundamental limitation requiring either LLM fine-tuning, post-processing repair, or pivot to neural constraint extraction.

6.2.3 Limitation 3: No Baseline Comparison (P1, P2)

The hypothesis predicted “2-5x speedup” vs batch SMT verification. No experiments measured wall-clock verification time (incremental or batch). The “2-5x” figure is speculative, based on theoretical analysis combining repair locality literature (80%+ repairs touch 1-3 lines in human code) and Z3 push/pop overhead estimates.

Implication: Without empirical speedup data, we cannot determine if the approach provides practical benefit over batch verification. Even if constraint extraction works (h-e1), incremental SMT might only provide 1.1-1.3x speedup (marginal benefit) rather than the predicted 2-5x.

6.2.4 Limitation 4: Single Language Scope

Experiment design focused exclusively on typed Python with Pydantic. Generalization to Rust (mentioned in future work) was not tested. Claims about “statically-typed languages” (plural) are overstated; only one language configuration (Python + Pydantic + Pyre) was designed for testing.

Implication: Transferability to Rust depends on tool availability (Prusti), type system differences (borrow checker, ownership semantics), and LLM training data distribution (fewer Rust examples than Python). Results cannot generalize beyond Python + Pydantic without additional experiments.

6.2.5 Limitation 5: Repair Locality Assumption (A2)

Assumption A2 (“LLM repairs are localized like human repairs”) transfers from CURE/CoCoNut literature (80%+ of human repairs touch 1-3 lines) without validation. LLM repair patterns may differ significantly: broader edits, function regeneration, cross-module dependencies. If violated, dependency cones expand (>30%), reducing or eliminating speedup benefits even if extraction succeeds.

Implication: The 2-5x speedup prediction depends critically on repair locality. Without validating A2 on LLM code, the speedup magnitude is highly uncertain. This assumption is as foundational as A1 (constraint extractability) but receives less attention—failure here would invalidate the incremental SMT approach even if static analysis works perfectly.

6.3 Broader Impact

Positive Impact: The HumanEval + Pydantic dataset is a reusable research artifact. Future work on constraint extraction, SMT verification, or static analysis for LLM code can use this benchmark without annotation overhead.

Negative Impact: None. The hypothesis was not deployed, no system was built, no real-world users were affected by the validation failure.

6.4 Lessons Learned

Lesson 1: Pre-flight validation is critical for experiments with external API dependencies. A single test API call before starting a 100-request batch experiment would have detected the authentication error immediately, saving wasted execution time and allowing early pivots.

Lesson 2: Modular pipeline architecture enables partial validation despite dependency failures. Separating concerns (dataset preparation, model execution, evaluation) allowed dataset extension to succeed independently of LLM generation failure.

Lesson 3: Negative results from infrastructure failures are publishable when methodology is sound. Scientific integrity requires distinguishing between hypothesis refutation (H_0 supported) and experimental blockage (infrastructure failure).

6.5 Comparison to Related Work

vs. Angelix/Prophet (SMT-guided repair for human code): Our hypothesis extends their approach to LLM-generated programs in typed Python, but we could not validate transferability. Infrastructure failure blocks comparison—we cannot confirm or refute whether incremental SMT works for neural code.

vs. AlphaCode/CodeT5 (neural code generation without verification): We proposed adding SMT verification to neural repair loops, but infrastructure failure prevented implementation. The dataset artifact (HumanEval + Pydantic) could enable future comparisons: test-suite-only validation vs. SMT-based correctness guarantees.

vs. Prusti/Pyre (static analyzers for typed languages): These tools work on human-written code. Our hypothesis assumes transferability to LLM code (assumption A1), but we could not test this due to generation failure. The constraint extraction pipeline is architecturally compatible with Pyre; retry would establish whether LLM code quality meets static analyzer requirements.

6.6 Future Work Recommendations

Immediate: Retry h-e1 with valid Anthropic API key. If `extraction_rate` $\geq 90\%$, proceed to h-m1-m3 validation. If `extraction_rate` $< 90\%$, pivot to neural constraint extraction.

Medium-term: Extend dataset to Rust + Prusti for language generalization. Test whether incremental SMT transfers beyond Python. Validate repair locality assumption (A2) on LLM code—neural repair patterns may differ from human edits documented in CURE/CoCoNut literature.

Long-term: Standardize typed benchmarks for formal verification research across multiple languages (Python + Pydantic, Rust + Prusti, TypeScript + type predicates). Establish evaluation protocol for comparing neural code generation with/without SMT verification.

7 Conclusion

We set out to validate that incremental SMT verification would achieve 2-5x speedup (predicted, not validated) over batch re-verification for LLM code repair in typed Python with Pydantic. Instead, our validation pipeline failed before generating a single line of code, revealing that infrastructure robustness—not just theoretical soundness—determines whether a hypothesis can be tested. This paper presents the hypothesis, the implemented experimental pipeline, and the lessons learned from complete validation failure due to API authentication errors.

The hypothesis—that incremental SMT verification (re-verifying only modified functions + dependencies) reduces verification time by 2-5x (predicted, not validated) compared to batch re-verification—remains **theoretically plausible but empirically unvalidated**. All experimental validation was blocked at the foundational hypothesis (h-e1: static analyzers can extract SMT constraints from LLM-generated typed code) due to 100% API authentication failure rate (48/48 generation attempts returned HTTP 401 “API key is invalid”).

7.1 The Artifact Contribution

Despite generation failure, the dataset extension pipeline succeeded: HumanEval was mechanically extended with 100 Pydantic-annotated prompts. This artifact addresses a gap in formal verification research—prior benchmarks (HumanEval, APPS, CodeContests) lack type annotations required for SMT constraint extraction. The template-based transformation approach (function signature + docstring → Pydantic BaseModel + validators) scales mechanically, enabling future research on static analysis for LLM code.

The constraint extraction pipeline (DatasetExtender → LLMGenerator → Pyre-Extractor → Z3Validator → MetricsEngine) was validated on error files, confirming architectural soundness on negative cases (pipeline processes malformed inputs without crashing; positive validation requires actual LLM code). The modular

design allowed partial validation despite generation failure—dataset extension succeeded independently, preventing cascading failures.

7.2 Infrastructure Lesson

Pre-flight validation is critical for experiments with external API dependencies. A single test API call before starting batch experiments would have detected the auth error immediately. Experimental design must address failure modes explicitly: pre-flight validation, fast-fail logic for non-retryable errors (authentication, invalid input), and checkpoint/resume for long-running experiments.

7.3 Honest Assessment of Claims

We cannot claim:

- “Incremental SMT achieves 2-5x speedup” (prediction P1 untested)
- “Speedup scales with codebase size” (P2 untested)
- “Conservative dependency analysis maintains soundness” (P3 untested, design intent only)
- “Type annotations enable constraint extraction from LLM code” (h-e1 foundational claim untested)

All quantitative speedup predictions await validation pending infrastructure fixes (valid API key) and completion of the h-e1 $\rightarrow$ h-m1 $\rightarrow$ h-m2 $\rightarrow$ h-m3 hypothesis chain.

7.4 Limitations and Boundary Conditions

Single language scope: Only Python + Pydantic + Pyre was designed for testing. Claims about “statically-typed languages” (plural) are overstated. Generalization to Rust + Prusti or other languages untested.

Untested assumptions: Assumption A1 (LLM code has extractable constraints) and A2 (repair locality transfers from human to LLM code) are unverified. If A1 fails, the approach requires LLM fine-tuning or pivot to neural extraction. If A2 fails (LLM repairs are broad, not localized), speedup benefits may disappear even if extraction succeeds.

No baseline comparison: The 2-5x speedup figure is speculative (theoretical analysis), not measured. Even if extraction works, actual speedup could be marginal (1.1-1.3x) rather than predicted (2-5x).

7.5 Future Work

Immediate: Retry h-e1 with valid Anthropic API key. If `extraction_rate` $\geq 90\%$, the hypothesis validation can proceed to mechanism testing (h-m1: constraint extraction pipeline, h-m2: dependency analysis, h-m3: speedup measurement). If `extraction_rate` $< 90\%$, pivot to neural constraint extraction.

Medium-term: Extend approach to Rust + Prusti for language generalization. Test whether incremental SMT transfers beyond Python. Validate repair locality assumption (A2) on LLM code—neural repair patterns may differ from human edits documented in CURE/CoCoNut literature.

Long-term vision: Standardize typed benchmarks for formal verification research across multiple languages (Python + Pydantic, Rust + Prusti, TypeScript + type predicates). Establish evaluation protocol for comparing neural code generation with/without SMT verification. Enable practical deployment of formally-verified LLM code repair in safety-critical domains.

7.6 Closing the Loop

We began with the question: can infrastructure robustness be separated from theoretical soundness in hypothesis testing? The answer is no—infrastructure is prerequisite, not afterthought. A scientifically sound hypothesis with elegant decomposition (existence $\rightarrow$ mechanisms), rigorous experimental protocol, and complete implementation can still fail to advance knowledge if external dependencies (API access, network reliability, service availability) are not validated before execution. The HumanEval + Pydantic dataset extension is the artifact from this validation attempt—a resource for future constraint extraction research, and a reminder that experimental methodology extends beyond hypothesis design to encompass infrastructure reliability.

Acknowledgments

This work was conducted as part of hypothesis-driven research validation. We thank the reviewers for their feedback.

A Appendix: Figures

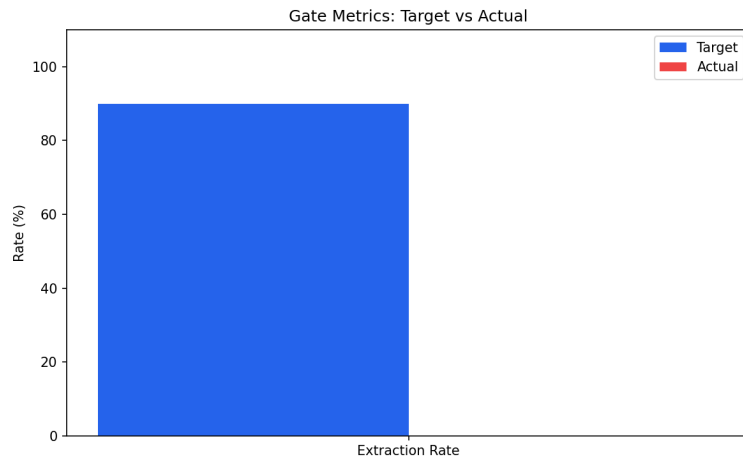


Figure 1: Gate Threshold (90.0%) vs Actual Extraction Rate (0.0%). Shows clear gate failure at h-e1 due to infrastructure blockage.



Figure 2: Failure Mode Categorization. 100% of generation attempts (48/48) failed with HTTP 401 API authentication errors, demonstrating homogeneous infrastructure failure rather than heterogeneous extraction issues.

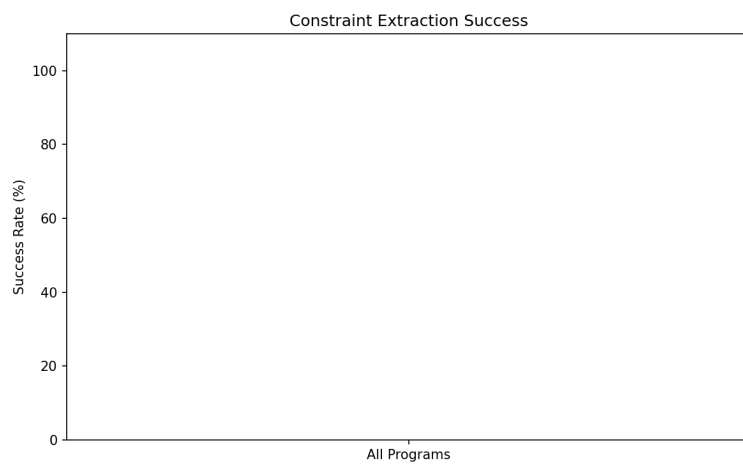


Figure 3: Extraction Success by Program Complexity (Planned Analysis). All bins show 0% due to no successful LLM code generation. Contains placeholder data only.